Sem IV	OE	Open Elective 1 (From Other School) Open-Source Technologies Introduction to Data Science	3	0	0	3	3
	PC	Design Thinking	0	0	2	2	1
	PC	Probability and Statistics Theory	3	0	0	3	3
	PC	Computer Organization and Architecture	3	1	0	4	4
	PC	Software Engineering	3	0	0	3	3
	PC	Software Engineering Laboratory	0	0	2	2	1
	PC	Theory of Computation	3	1	0	4	4
	PC	Design and Analysis of Algorithm	3	0	0	3	3
	PC	Design and Analysis of Algorithm Laboratory	0	0	2	2	1
			18	**2**	**6**	**24**	**23**
Sem V	OE	Open Elective 2 ((NPTEL/SWAYAM/MOOC))	3	0	0	3	3
	ESC	Introduction to Artificial Intelligence (Department Specific)	3	0	0	3	3
	HSC	Engineering Economics	3	0	0	3	3
	PE	Program Elective 1	3	0	0	3	3
	PC	Computer Networks	3	0	0	3	3
	PC	Computer Networks Laboratory	0	0	2	2	1
	PC	Compiler Design	3	0	0	3	3
	PC	Compiler Design Laboratory	0	0	2	2	1
	PC	Operating System	3	0	0	3	3
	PC	Operating System Laboratory	0	0	2	2	1
			21	**0**	**6**	**27**	**24**
Sem VI	OE	Open Elective 3 (From Other Department of FoET) Web Development Object Oriented Programming & Design	3	0	0	3	3
	PE	Program Elective 2	3	0	0	3	3
	PE	Program Elective 3	3	0	0	3	3
	PC	Cryptography and Network Security	3	0	0	3	3
	PC	Cryptography and Network Security Laboratory	0	0	2	2	1
	PC	Web & Mobile Development Essentials	1	0	0	1	1
	PC	Web & Mobile Development Essentials Laboratory	0	0	4	4	2
	PC	Distributed Computing	3	0	0	3	3
	PC	Distributed Computing Laboratory	0	0	2	2	1
			16	**0**	**8**	**24**	**20**
Sem VII	Pro	Summer Internship	0	0	0	0	2
	OE	Open Elective 4 (From Other Department of FoET) Green IT	3	0	0	3	3

4th Semester

24CS221T					Open Source Technologies					
Teaching Scheme					Examination Scheme					
L	T	P	C	Hrs/Week	Theory			Practical		Total Marks
					MS	ES	IA	LW	LE/Viva	
3	0	0	3	3	25	50	25	--	--	100

COURSE OBJECTIVES

- To create a strong foundation of the individual components of web development.
- To manage the configuration and optimization of every LAMP stack component.
- To learn database design principles, SQL queries, and database management techniques.
- To develop proficiency in server-side scripting and building dynamic web applications.

UNIT 1: Linux Operating System Introduction to Linux Operating System, Linux File System Hierarchy, Essential commands for navigating the file system, File and directory manipulation commands, Understanding file permissions (read, write, execute) and ownership, Using chmod and chown commands to modify permissions and ownership, Shell scripting basics, Overview of shell scripting and its importance, Writing and executing simple shell scripts.	**10 Hrs.**
UNIT 2: Apache Web Server Introduction to Apache web server, Installing apache on Linux platforms, Apache configuration file structure, Apache log files, Apache-related commands, Starting apache for the first time PHP Installation and Configuration. Current and Future version of PHP/MySQL, Installing / Building PHP/MySQL on different platforms with Apache.	**10 Hrs.**
UNIT 3: MySQL Database Introduction to MySQL Database, MySQL database management, installation, configuration, database design, MySQL datatypes, SQL querying, Data definition language, Data manipulation language, Transaction and stored procedures in MySQL, Administration tasks and integration with programming languages.	**10 Hrs.**
UNIT 4: PHP – Server side language Introduction to PHP, PHP Syntax and Variables, Control Structures, Functions and Arrays, String Manipulation and Regular Expressions, Form Handling and Super Global Variables, File Handling, Error Handling and Exception Handling, Object-Oriented Programming in PHP, Database Connectivity with MySQL, Session Management and Cookies, Web Application Development.	**12 Hrs.**
	42 Hrs.

COURSE OUTCOMES

On completion of the course, student will be able to

CO1 - Identify the role of each component (Linux, Apache, MySQL, PHP) in the open-source technologies.

CO2 - Understand the impact of Linux file system security within the LAMP stack environment.

CO3 - Illustrate the process of handling HTTP requests and responses using Apache web server.

CO4 - Interpret SQL queries to retrieve and manipulate data stored in MySQL databases.

CO5 - Apply object-oriented programming principles in PHP to develop scalable and maintainable web applications.

CO6 - Analyze database schemas and query execution plans to optimize database performance and query efficiency in MySQL.

TEXT/REFERENCE BOOKS

1. James Lee, "Open Source Development with LAMP: Using Linux, Apache, MySQL, Perl, and PHP", Pearson.
2. Jon Duckett, "PHP & MySQL: Server-side Web Development" Wiley.
3. Robin Nixon, "Learning PHP, MySQL & JavaScript: A Step-by-Step Guide to Creating Dynamic Websites (Learning PHP, MYSQL, Javascript, CSS & HTML5)", O'Reilly Media.

24CS222T					Introduction to Data Science					
Teaching Scheme					Examination Scheme					
L	T	P	C	Hrs/Week	Theory			Practical		Total Marks
					MS	ES	IA	LW	LE/Viva	
3	0	0	3	3	25	50	25	--	--	100

COURSE OBJECTIVES

- Understand issues relating to acquisition, cleaning and loading of data,
- Be able to perform exploratory data analysis using Python,
- Understand the basics of how data can be presented and visualized,
- Become familiar with some very fundamental ethical issues in data science.

UNIT 1 Introduction to Data and Science behind the data Introduction to Data Science, Toolboxes: Python, fundamental libraries for data Scientists. Integrated development environment (IDE). Data operations: Reading, selecting, filtering, manipulating, sorting, grouping, rearranging, ranking, and plotting.	**10 Hrs.**
UNIT 2 Data Collection, Pre-Processing and EDA Data Collection Strategies, Data Pre-Processing Overview, Data Cleaning, Data Integration and Transformation, Data Reduction, Data Discretization, Descriptive statistics, data preparation. Exploratory Data Analysis data summarization, data distribution, measuring asymmetry. Sample and estimated mean, variance and standard score. Statistical Inference frequency approach, variability of estimates, hypothesis testing using confidence intervals, Box Plots, Pivot Table, Heat Map, Correlation Statistics.	**12 Hrs.**
UNIT 3 Model Development Supervised Learning: First step, learning curves, training-validation and test. Learning models generalities, Regression techniques: Linear/Polynomial regression, Classifications: Naïve Bayes, Logistic Regression, support vector machines	**12 Hrs.**
UNIT 4 Model Evaluation Generalization Error, Out-of-Sample Evaluation Metrics, Cross Validation, Overfitting, Under Fitting and Model Selection, Prediction by using Ridge Regression, Testing Multiple Parameters by using Grid Search, precision, recall, F1 score.	**8 Hrs.**
	42 Hrs.

COURSE OUTCOMES

On completion of the course, student will be able to

CO1 - Understand the fundamental of data science concepts, including Python toolboxes and basic data operations.

CO2 - Understand the principles of supervised learning including its applications, advantages, and limitations.

CO3 - Apply data pre-processing techniques, including data cleaning, integration, transformation, and reduction, to prepare raw data for analysis.

CO4 - Apply statistical inference methods to draw conclusions and make decisions from data analysis.

CO5 - Construct supervised learning models, including regression and classification techniques

CO6 - Demonstrate the techniques to solve real-world data science problems effectively.

TEXT/REFERENCE BOOKS

1. Joel Grus, Data Science from Scratch, O'Reilly Media, Inc.
2. Cathy O'Neil, Rachel Schutt, Doing Data Science, Straight Talk from The Frontline. O'Reilly Media Inc.
3. Introducing Data Science, Davy Cielen, Arno D. B. Meysman, Mohamed Ali, Manning Publications Co.
4. Storytelling with Data: A Data Visualization Guide for Business Professionals,v Nussbaumer Knaflic, Wiley
5. Time Series Analysis 1st Edition, James D. Hamilton, Princeton University Press.

24CS205T					Design Thinking Laboratory					
Teaching Scheme					Examination Scheme					
L	T	P	C	Hrs/Week	Theory			Practical		Total Marks
					MS	ES	IA	LW	LE/Viva	
0	0	2	1	2	-	-	-	50	50	100

COURSE OBJECTIVES

- To immerse students in contemporary perspectives, ideas, concepts, and solutions pertinent to designing and executing projects utilizing principles of design thinking.
- To cultivate a mindset and discipline of systemic inspiration driven by a curiosity to explore novel sources of ideas and unconventional models, particularly beyond their typical professional environments.
- To formulate a concrete, feasible, viable, and relevant innovation project or challenge.

LIST OF EXPERIMENTS:

1. **Introduction to Design Thinking:**
   - Differentiating Design Thinking, Principles of Design Thinking
   - The Basis for Design Thinking, The Design Thinking Team
2. **Design Thinking Workshop Essentials**
   - Design Thinking Workshops and Meetings
   - Exercises and Case-based Discussions
3. **Listening and Empathizing Techniques**
   - Observation Skills
   - Structured Open-Ended Approaches, Exercises and Case-based Discussions
4. **Ideation Tools and Frameworks**
   - Design Thinking Frameworks
   - Ideation Tools like Brainstorming and Innovation Heuristics
   - Overcoming Cognitive Fixedness
5. **Using Diagrams and Maps in Design Thinking**
   - Empathy Map, Affinity Diagram, Mind Map, Journey Map
   - Combining Ideas into Complex Innovation Concepts
6. **Storytelling and Scenario Planning**
   - Improvisation Techniques, Scenario Planning
   - Development and Evaluation of Scenarios, Frog Design and Prototyping
7. **Assessing Bias and Strengthening Communication**
   - Assessing Developer and User Perspectives for Bias
   - Applying Frameworks to Strengthen Communication and sustaining a Culture of Innovation

COURSE OUTCOMES

On completion of the course, student will be able to

CO1- Identify different stakeholders involved in a design thinking project.
CO2- Describe how empathy plays a crucial role in the design thinking process.
CO3- Implement listening and empathizing techniques to understand user needs effectively.
CO4- Assess biases in developer and user perspectives to strengthen communication and foster innovation.
CO5- Evaluate the efficacy of storytelling in communicating design solutions.
CO6- Utilize user observations to create empathy maps and journey maps.

TEXT/REFERENCE BOOKS

1. Roger Martin, "The Design of Business: Why Design Thinking is the Next Competitive Advantage",Harvard Business Press.
2. Hasso Plattner, Christoph Meinel and Larry Leifer, "Design Thinking: Understand –Improve– Apply", Springer.
3. Idris Mootee, "Design Thinking for Strategic Innovation: What They Can't Teach You at Business or Design School", John Wiley & Sons.

24CS206T					Probability and Statistics					
Teaching Scheme					Examination Scheme					
L	T	P	C	Hrs/Week	Theory			Practical		Total Marks
					MS	ES	IA	LW	LE/Viva	
3	0	0	3	3	25	50	25	-	-	100

PREREQUISITES: Basic knowledge of linear algebra, calculus, and mathematical reasoning.

COURSE OBJECTIVES

- To introduce the fundamental concepts and theorems of probability theory.
- Calculate and interpret measures of central tendency and dispersion for different distributions.
- Apply elements of statistical processes for the solution of real life problems.
- Interpret confidence intervals and hypothesis tests to make inferences about population parameters.

UNIT 1: Fundamentals Of Probability Sample space, Axiomatic definition and properties, Probability set function, Conditional probability, Bayes' rule and independence of events, Random variables, Distribution function, Probability mass and density functions, Expectation, Variance, moments, Moment generating function, Chebyshev's inequality.	**10 Hrs.**
UNIT 2: Random Variable Distributions Uniform, Bernoulli, Binomial, Geometric, Negative binomial, Hypergeometric, Poisson, Exponential, Gamma, Weibull, Beta, Cauchy, Double exponential, Normal, Joint distributions, Marginal and Conditional distributions, Weak Law of large numbers and Central limit theorems.	**11 Hrs.**
UNIT 3: Statistical Inference Sampling Techniques and Estimation: Population, Sample, Parameters, Random sampling with and without replacement, Point estimation, Maximum likelihood estimation (MLE) and method of moments, Confidence intervals for population parameters, Hypothesis Testing: Null and alternative hypotheses, Type I and Type II errors, Common hypothesis tests: t-tests, chi-squared tests, and ANOVA.	**11 Hrs.**
UNIT 4: Curve Fitting And Regression Analysis Independence of random variables, Covariance, Functions of random variables, Measure of central tendency, Curve Fitting, Correlation, Simple Correlation, Partial Correlation, Linear regression, Multiple regression, Residual analysis and diagnostics.	**10 Hrs.**
	42 Hrs.

COURSE OUTCOMES

On completion of the course, student will be able to

CO1 - Understand the basic principles of probability theory, including sample spaces, events, and probability axioms.

CO2 - Explain the concept of statistical inference and its role.

CO3 - Apply statistical techniques to real-world data sets using software tools.

CO4 - Implement linear and logistic regression models for predictive analytics.

CO5 - Assess the reliability of statistical inference methods based on probability theory, such as confidence intervals and hypothesis testing.

CO6 - Create and justify mathematical models to represent complex probabilistic phenomena and analyze their implications for decision-making processes.

TEXT/REFERENCE BOOKS

1. Robert V. Hogg, Joseph W. McKean and Allen T. Craig, “Introduction to Mathematical Statistics”, Pearson Education.
2. Paul G. Hoel, Sidney Port, and Charles Stone, “Introduction to Probability Theory”, Houghton Mifflin.
3. K. Rohatgi and A. K. Md. E. Saleh: An Introduction to Probabilityand Statistics, John Wiley & Sons.
4. Boes, D. C., Graybill, F. A., Mood, A. M., Introduction to the Theory of Statistics. McGraw-Hill International Book Company.
5. Ross, Sheldon M., Introduction to Probability Models, Elsevier Inc. AP.

24CS207T					Computer Organization and Architecture					
Teaching Scheme					Examination Scheme					
L	T	P	C	Hrs/Week	Theory			Practical		Total Marks
					MS	ES	IA	LW	LE/Viva	
3	1	0	4	4	25	50	25	--	--	100

PREREQUISITE : Digital Logic Design

COURSE OBJECTIVES

- To introduce fundamental concepts and principles underlying computer architecture.
- To explore the components of a computer system, including the CPU, memory, I/O devices, and bus architecture.
- To study the memory hierarchy, understand their organization, performance characteristics, and management techniques.
- To examine the design principles of computer arithmetic units.
- To explore techniques for optimizing computer system performance.

UNIT 1: Processor Architecture Fundamentals X86 Architecture, CPU Registers, Segment Registers, General Purpose Registers, Bus Interface Unit, Address bus, Data bus and control bus, Control Unit, ALU, Memory unit, Architectural overview and comparison between 32-bit and 64-bit processor, Microprocessor Pin Configuration, Microcontroller, Comparison with microprocessor, AI Processor, comparison of AI Processor with regular Microprocessor, ARM processor Architecture, Instruction Set Architecture, Instruction interpretation and execution, MIPS Instructions format, ARM – Instruction Set Architecture.	**10 Hrs.**
UNIT 2: ALU, Micro-operation and RTL Arithmetic and logic unit, Register configuration of Signed Magnitude Addition-Subtraction Algorithm, Register configuration of 2's complement Addition Subtraction Algorithm, Signed Magnitude Multiplication Algorithm, Booth's multiplication algorithm, Addressing Modes, Immediate, Direct, Indirect, PC-relative, Indexed, Subroutine Call, Micro-operation and their RTL specification.	**9 Hrs.**
UNIT 3: Control Unit and Memory Architecture CPU control unit design: Hardwired and micro-programmed design approaches, design of a simple hypothetical CPU. CPU – Memory interconnections, Organization of memory modules, Associative memory, Cache Memory, Memory Mapping Techniques. Peripheral devices and their characteristics: Input-output subsystems, I/O device interface, I/O transfers – program controlled, interrupt driven and DMA, privileged and non-privileged instructions, software interrupts and exceptions. Programs and processes – role of interrupts in process state transitions, I/O device interfaces – SCII, USB, Secondary Storage – solid-state drive (SSD), Comparison of SSD with HDD.	**12 Hrs.**
UNIT 4: Pipelining and Multiprocessing Unit Pipelining, pipeline hazards, Design issues of pipeline architecture, Instruction level parallelism and advanced issues, RISC and CISC paradigm, Design issues of a RISC processor, Multiprocessor system and its characteristics, Advance Research Topics in Microprocessor, Parallel processing concepts, Parallelism algorithm for multiprocessor systems, Introduction to GPU Architecture, Comparison between GPU and CPU Architecture, Interconnection network, Cache in multiprocessor systems and related problems, Cache coherence protocols.	**11 Hrs.**
	42 Hrs.

TUTORIALS

UNIT 1:	• Case Studies to differentiate between 32-bit and 64-bit processors. • Exploring Bus interface unit, control unit, segment registers, and general-purpose registers. • Case studies and examples showcasing real-world applications of microcontrollers.	**3 Hrs.**

UNIT 2:	• Real-world examples and applications of different addressing modes in assembly language programming. • Numerical problem solving on different addressing modes. • Exercises to implement subroutine calls using various addressing modes.	**3 Hrs.**
UNIT 3:	• Comparative analysis of hardwired and micro-programmed design approaches in terms of complexity, flexibility, and performance. • Case studies on CPU-memory interconnections and organization of memory modules. • Problem solving on Memory organization. • Real-world examples and case studies illustrating the role of interrupts in process state transitions.	**4 Hrs.**
UNIT 4:	• Examples on pipelining and its advantages in CPU performance. • Exercises on identifying and mitigating pipeline hazards in sample code segments. • Case studies and examples illustrating the differences between RISC and CISC instruction sets. • Exercises on parallel algorithms on multiprocessor systems.	**4 Hrs.**

COURSE OUTCOMES

On completion of the course, student will be able to

CO1 - Understand fundamental computer architectures, identifying their strengths, weaknesses, and trade-offs.

CO2 - Classify basic computer arithmetic units, including adders, multipliers, and dividers, using digital logic circuits.

CO3 - Apply knowledge of instruction set architecture to influence the design and functionalities of CPU.

CO4 - Apply the memory management techniques in improvement of computer performance.

CO5 - Differentiate the performance of computer systems using techniques such as pipelining, caching, and branch prediction.

CO6 - Apply critical thinking and problem-solving skills to manage real-world design challenges in computer architecture.

TEXT/REFERENCE BOOKS

1. J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, Morgan Kaufmann.
2. M. M. Mano, Computer System Architecture, Prentice Hall of India, New Delhi.
3. William Stallings, Computer Organization and Architecture, Pearson.
4. C. Hamacher, Z. Vranesic, S. Zaky, Computer Organization, McGraw Hill Education.
5. J. P. Hayes, Computer Architecture and Organization, McGraw Hill Education.

24CS209T					Software Engineering					
Teaching Scheme					Examination Scheme					
L	T	P	C	Hrs/Week	Theory			Practical		Total Marks
					MS	ES	IA	LW	LE/Viva	
3	0	0	3	3	25	50	25	--	--	100

PREREQUISITE: Object oriented programming concepts

COURSE OBJECTIVES

- To gain a deep understanding of the systematic approach to developing, operating, maintaining, and retiring software throughout its lifecycle.
- To exhibit strong communication and interpersonal skills, adhering to professional and ethical principles when working as both team members and leaders in multi-disciplinary teams.
- To apply software engineering principles for product development.

UNIT 1: Introduction and Essence of Modern Software Engineering Introduction, Characteristics of Software, need for software engineering, Software quality attributes, Software product pipelines, Software Development Life Cycle and Process: Waterfall, Iterative models, Exploratory models, RAD, Agile and Scrum, Requirement Analysis, Functional and non-functional requirements, the software requirements document and SRS standards, Requirements Engineering Process: Inception, Elicitation, Elaboration, Negotiation, Specification. Software engineering essence, Essence language, Essence kernel, Using essence kernel in agile development practices, Agile Principles, Agile process models through essence kernel, Large scale complex development Using kernel.	**12 Hrs.**
UNIT 2: Modelling & Design Design principles, Design Patterns, Architecture Versus Design, Modularity, Software Components and Connectors, Architecture Styles. System modelling and UML diagrams.	**10 Hrs.**
UNIT 3: Testing & Quality Management Software Testing Strategies: verification vs validation, Types of testing and sub-testing adopted for various software: unit, integration, validation, system, and acceptance testing, Software Quality Assurance vs control, The ISO 9000 quality standards, CMMI Framework, Test case generations, Cyclometric complexity.	**10 Hrs.**
UNIT 4: Software Project Management Maintenance & Reengineering, Risk management: Reactive vs Proactive Risk strategies, software risks, Risk identification, Risk projection, Risk refinement, RMMM, RMMM Plan. Software versioning and Continuous integration, Project management and Risk analysis, Configuration management, Cost analysis and estimation.	**10 Hrs.**
	42 Hrs.

COURSE OUTCOMES

At the end of the course, students will be able to –

CO1 - Understand Unified Modeling Language to specify, construct, and document the artifacts of software products.

CO2 - Understand procedural, non-procedural and object-oriented design approaches for efficient software product construction.

CO3 - Understand software testing life cycle and deduce manual test cases for software product.

CO4 - Identify software and hardware requirements of a project.

CO5 - Analyze software product risks and its management strategies.

CO6 - Assess software quality with ISO and CMMI standards.

TEXT/REFERENCE BOOKS

1. Roger S Pressman, Software engineering A practitioner's Approach, McGraw Hill
2. Ian Sommerville, Software Engineering, Pearson education.
3. Pankaj Jalote, Software Engineering, A Precise Approach, Wiley India.
4. Rajib Mall, Fundamentals of Software Engineering, PHI
5. Unified Modeling Language Reference Manual, James Rumbaugh, Ivar Jacobson, Grady Booch, Addison-Wesley Professional

24CS209P					Software Engineering Laboratory					
Teaching Scheme					Examination Scheme					
L	T	P	C	Hrs/Week	Theory			Practical		Total Marks
					MS	ES	IA	LW	LE/Viva	
0	0	2	1	2	-	-	-	50	50	100

PREREQUISITE: Object oriented programming concepts

COURSE OBJECTIVES

- To foster problem-solving skills and encourage critical thinking by challenging students with real-world software engineering problems and guiding them through the process of finding innovative solutions.
- To raise awareness about ethical and professional responsibilities in software engineering adhering to industry standards and best practices.
- To emphasize the importance of clear and concise documentation throughout the software development process, including requirements documentation, design documents, code comments, and user manuals.

LIST OF EXPERIMENTS:

2. **Requirements Analysis Using Case Modelling:**
   - Conduct interviews or surveys with potential users to gather software requirements.
   - Analyse gathered requirements to identify functional and non-functional requirements.
   - Actor definitions. Writing a case goal.
   - Create use case diagrams, user stories, and requirement documents based on the collected requirements.
8. **Transfer from Analysis to Design in the Characterization Stage**
   - Identify Method, Operation, Object Interface, Class.
   - Finding objects from Flow of Events.
   - Use UML (Unified Modelling Language) to create class diagrams, sequence diagrams, collaboration diagrams, component, activity, and deployment diagrams.
9. **Coding and Implementation.**
   - To Develop small software modules using programming languages like C, C++, Java, Python, or JavaScript.
   - Apply principles of clean code, such as SOLID to write maintainable and readable code.
   - Practice pair programming or code reviews to improve code quality and learn from peers.
10. **Software Testing**
    - Write unit tests using testing frameworks like JUnit, Pytest, or Jasmine.
    - Perform integration testing by combining individual modules and testing their interactions.
    - Conduct system testing to verify that the software meets the specified requirements.
11. **Documentation and Presentation**
    - Create documentation including user manuals, API documentation, and technical specifications.
    - Prepare and deliver presentations to communicate project progress, findings, and recommendations.
    - Practice writing clear and concise technical reports summarizing project activities and outcomes.
12. **Group projects**
    - Participate in a simulated agile development process using Scrum or Kanban boards.
    - Adapt to changing requirements and prioritize tasks based on lab instructor's feedback during iterative development.

COURSE OUTCOMES

On completion of the course, student will be able to

CO1 - Recall fundamental principles and concepts of software engineering.

CO2 - Interpret UML diagrams and understand their significance in software design.

CO3 - Apply software design principles and design patterns to solve specific software design problems.

CO4 - Implement testing techniques to verify and validate software functionality.

CO5 - Synthesize project technical documentation and reports using UML design tools that effectively communicate project progress and outcomes.

CO6 - Design and collaboratively work on group projects to design software architectures and systems that meet specified requirements.

TEXT/REFERENCE BOOKS

4. The Unified Modelling Language User Guide. Grady Booch, James Rumbaugh, Ivar Jacobson, Pearson Education.
5. Object-Oriented Software Engineering: using UML, Patterns, and Java. Bernd Bruegge and Allen H. Dutoit.
6. Design Patterns: Elements of Reusable Object-Oriented Software. Erich Gamma, Richard Helm, Ralph Johnson, and John M. Vlissides.

24CS208T					Theory of Computation					
Teaching Scheme					Examination Scheme					
L	T	P	C	Hrs/Week	Theory			Practical		Total Marks
					MS	ES	IA	LW	LE/Viva	
3	1	0	4	4	25	50	25	-	-	100

PREREQUISITE: Data Structure, Discrete Mathematics

COURSE OBJECTIVES

- Understand automata as a basic model of computation and abstractions.
- Link between formal languages, grammars, automata, and decision problems.
- Understand the Classification of machines by their power/ limitations to recognize languages.
- Understand algorithms and computability through the lens of Turing machines.

UNIT 1: Finite Automata and Regular Language Regular Languages: Alphabets, Language, Regular Expression, Definitions of Finite State automata, Transition Graphs, Conversion of DFA to Regular Expression, Deterministic & Non-deterministic Finite State Automata, Regular Grammar, Regular languages: Closure properties, product construction, Equivalence of NFA and DFA, NFA with epsilon transitions, Thompson's Construction to Convert Regular Expression to NDFA, Subset Algorithm to convert NDFA to DFA, Minimization of DFA, Finite State Machine with output-Moore machine and Mealy Machine, Conversion of Moore machine to Mealy Machine & Vice-Versa, Limitations of Automata Non-regularity: Pumping Lemma and Myhill–Nerode theorem.	**12 Hrs.**
UNIT 2: Pushdown Automata and Context-free Language Context Free Grammar, Derivation tree and Ambiguity, Removing ambiguity, Parse Tress, Application of Context free Grammars, Closure Properties of CFG, Decidable of CFG, Push Down Automata: definition, the Languages of a PDA, Equivalence of PDAs and CFGs, Equivalence of PDAs accepting by final state and by empty stack, Non-determinism in PDAs, Non-equivalence of NPDAs and DPDAs, Pumping Lemma for Context-Free Languages, Ogden's lemma, Normal Forms for CFG's: Chomsky Normal Form and Greibach Normal form, Context–Sensitive Grammars, Linear Bounded Automata.	**12 Hrs.**
UNIT 3: Turing Machine and Recursive Language Universal Turing Machine, Post Machine, Deterministic Turing machines, Multi-tape TMs and its equivalence to single-tape TMs, Nondeterministic TMs and its equivalence to deterministic TMs, Chomsky Hierarchy, Decidable and recognizable languages, Post correspondence problem, Halting Problem, Turing enumerability, Turing machines Acceptability, Decidabilities, and Reducibility, Unsolvable problems about Turing machines, Rice's Theorem, The Church-Turing Thesis.	**09 Hrs.**
UNIT 4: Computability and Complexity The Extended Church-Turing Thesis, Time Complexity, P and NP complexity class, SAT, Polynomial time Reducibility, The time hierarchy theorem, Definition of NP via non-deterministic Turing machines, NP-complete problems, Cook-Levin theorem, NP-complete problems via reductions, PSPACE complexity class, PSPACE-Completeness, PSPACE = NPSPACE, BPP complexity class and relation with the other classes.	**09 Hrs.**
	42 Hrs.

TUTORIALS

1. Problems DFA construction, and conversion of DFA to Regular Expression.
2. Problems on NFA construction and epsilon transitions.
3. Problems on NFA to DFA Conversion.
4. Problems on Minimization of DFA.
5. Problems on Mealy and Moore machine construction, and conversion.
6. Pumping Lemma and Problems on Grammar construction.
7. Problems on parsing and ambiguity removal.
8. Problems on PDA construction, accepted by final state and by empty stack.
9. Problems on NPDA construction and Pumping Lemma for CFL.
10. Conversion from CNF to GNF.
11. Problems on Deterministic and Non-Deterministic TM Construction.

12. Problems on Multi-tape TM and its equivalence conversion.
13. Problems on TM acceptability, decidabilities, and reducibility.
14. Problems on SAT, Polynomial time reducibility, and time hierarchy theorem.

COURSE OUTCOMES

On completion of the course, student will be able to

CO1 - Define key terms and concepts in automata theory, formal languages, and computability theory.
CO2 - Understand the finite state machines for modelling and solving computing problems.
CO3 - Construct finite automata for specific regular languages and prove the properties of regular languages.
CO4 - Analyse context-free grammar to design pushdown automata.
CO5 - Apply the concepts of Turing machine to understand decidability and undecidability.
CO6 - Analyse the probabilistic Turing machine and its bounded error class.

TEXT/REFERENCE BOOKS

1. John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, Introduction to Automata Theory, Languages, and Computation, Pearson Education.
2. Mishra and Chandrashekaran, Theory of Computer Science – Automata languages and computation, PHI.
3. Michael Sipser, Introduction to the Theory of Computation, Cengage Learning.
4. John C Martin, Introduction to Languages and the Theory of Computation, TMH.
5. Sanjeev Arora and Boaz Barak, Computational Complexity: A Modern Approach; Cambridge.

24CS210T					Design and Analysis of Algorithms					
Teaching Scheme					**Examination Scheme**					
L	**T**	**P**	**C**	**Hrs/Week**	**Theory**			**Practical**		**Total Marks**
					MS	**ES**	**IA**	**LW**	**LE/Viva**	
3	**0**	**0**	**3**	**3**	**25**	**50**	**25**	**--**	**--**	**100**

COURSE OBJECTIVES

- Analyse the asymptotic performance of the algorithms
- Implement time and space efficient optimized algorithms.
- Demonstrate a familiarity with major algorithms and data structures.
- Apply important algorithmic design paradigms and methods of analysis.

UNIT 1: Introduction Elementary Algorithmic concept: Efficiency of Algorithms, Average & worst-case analysis, Elementary Operation Analysis Techniques. Analyzing control structures, Amortized analysis.	**8 Hrs.**
UNIT 2: Recurrence Solving And Divide & Conquer **Recurrence solving**: Intelligent guesswork and proof by induction, change of variable, recurrence tree, master theorem. **DnC**: Binary Search, Merge sort, multiplying large integers, finding the median, Matrix multiplication, Exponentiation	**12 Hrs.**
UNIT 3: Greedy Algorithms And Dynamic Programming **Greedy**: Coin change, Interval scheduling, interval partitioning, job sequencing with deadline, fractional knapsack, MST using Kruskal algorithm, Dijkstra's algorithm, Huffman coding. **Dynamic**: Coin change, 0/1 Knapsack, Floyd-Warshall algorithm, Matrix chain multiplication, Assembly line scheduling, Travelling Salesman Problem.	**12 Hrs.**
UNIT 4: Backtracking, Branch & Bound, Intractability **Backtracking**: 0/1 Knapsack, n-queens problem. **Branch and Bound**: 0/1 Knapsack, Travelling Salesman Problem. **Intractability**: Brief Overview of NP theory, Some NPC problems: Independent set, vertex cover, clique, Hamiltonian cycle; Polynomial time reduction: Reduction of 3 SAT to clique	**10 Hrs.**
	42 Hrs

COURSE OUTCOMES

After completion of the course, students will be able to:

CO1 - Understand need of complexity analysis of the algorithm.

CO2 - Solve Recurrence relations using Master Theorem, Substitution method, and Recurrence tree.

CO3 - Apply different algorithmic strategies to solve computational problems.

CO4 - Identify the appropriate algorithmic strategy for the given problem definition and showcase proficiency in problem solving.

CO5 - Evaluate Classical problems through Backtracking and Branch & Bound techniques.

CO6 - Analyse the complexity of problem classes P, NP, NP-Complete and NP-Hard.

TEXT/REFERENCE BOOKS

1. Charles E. Leiserson, Thomas H. Cormen, Ronald L. Rivest, Clifford Stein - Introduction to Algorithms, MIT Press.
2. Gilles Brassard & Paul Bratley, Fundamentals of Algorithmic, PHI
3. Ellis Horowitz, Sartaj Sahni, Sanguthevar Rajasekharan, Fundamentals of Computer Algorithms, Galgotia.

24CS210P					Design and Analysis of Algorithms Laboratory					
Teaching Scheme					Examination Scheme					
L	T	P	C	Hrs/Week	Theory			Practical		Total Marks
					MS	ES	IA	LW	LE/Viva	
0	0	2	1	2	-	-	-	50	50	100

COURSE OBJECTIVES

- Analyse the asymptotic performance of the algorithms
- Implement time and space efficient optimized algorithms.
- Demonstrate a familiarity with major algorithms and data structures.
- Apply important algorithmic design paradigms and methods of analysis.

LIST OF EXPERIMENTS

Basic Programs and their Analysis:

- Selection sort, insertion sort, merge sort, quick sort.
- Linear and binary search.

Greedy:

- Implement fractional knapsack.
- Implement Kruskal's method for MST using union find data structures.
- Implement Interval scheduling.
- Implement Dijkstra's algorithm for single source shortest path.

Divide and Conquer:

- Implement exponential function.
- Implement the Strassen's matrix multiplication.
- Implement program to multiply two large integers.

Dynamic:

- Implement assembly line scheduling.
- Implement chained matrix multiplication.
- Implement Floyd Warshall algorithm for all pair shortest path problem.
- Implement the 0/1 knapsack problem.

Backtracking and Branch & Bound:

- Implement n-queens problem using backtracking
- Implement TSP using branch and bound.

COURSE OUTCOMES

After completion of the course, students will be able to:

CO1 - Understand need of complexity analysis of the algorithm.

CO2 - Apply greedy strategies to solve standard problems.

CO3 - Apply divide and conquer strategies to solve traditional problems.

CO4 - Apply dynamic programming to solve computational problems.

CO5 - Evaluate classical problems through backtracking and branch & bound techniques.

CO6 - Analyse the complexity of problem classes P, NP, NP-Complete and NP-Hard.

TEXT/REFERENCE BOOKS

1. Charles E. Leiserson, Thomas H. Cormen, Ronald L. Rivest, Clifford Stein - Introduction to Algorithms, MIT Press.
2. Gilles Brassard & Paul Bratley, Fundamentals of Algorithmic, PHI.
3. Ellis Horowitz, Sartaj Sahni, Sanguthevar Rajasekharan, Fundamentals of Computer Algorithms, Galgotia.